

A Project Report on

SMART MEDICATION SYSTEM

Bachelor of Technology

in

Computer Science and Engineering

Asmi Pandey | Roll No.: R9PV33A103 | Reg. No.: 12418814

Under the Guidance of

Deepak Kumar & Mahipal Singh Papola

Department of Computer Science and Engineering
Lovely Professional University, Punjab, India

Academic Session: 2026–2027

Table of Contents

S. No.	Contents	Page No.
1	Introduction	03
	1.1 Problem Statement	03
	1.2 Objectives	04
	1.3 Project Modules	04
	1.4 Technologies Used	05
2	Module Description	05-06
3	Implementation	07
	3.1 Data Structures / Algorithms Used	07-08
	3.2 Important Source Code with Explanation	08-10
4	Screenshots of the Project	11-13
5	Future Scope of the Project	13-14
6	Conclusion	15
7	References	15

1. Introduction

Healthcare has become one of the most important aspects of modern life, and proper medication management plays a significant role in maintaining good health. Many individuals, especially elderly people and patients suffering from chronic diseases, are required to consume multiple medicines every day at specific time intervals. Managing these medicines manually can become difficult because people often forget medicine schedules, fail to monitor medicine stock, or continue using medicines beyond their expiry dates. Such mistakes may lead to serious health complications and reduce the effectiveness of medical treatment.

Traditionally, people maintain medicine information using notebooks, paper prescriptions, or by relying on memory. These traditional methods are not efficient because they do not provide automatic reminders, inventory tracking, or expiry monitoring. As the number of medicines increases, managing them manually becomes increasingly complex and time-consuming. Therefore, there is a need for a computerised system that simplifies medicine management and assists users in maintaining accurate medication records.

The Smart Medication System is a desktop-based application developed using Java Swing that provides an efficient and user-friendly solution for medicine management. The application enables users to store medicine details, monitor medicine inventory, receive information about upcoming medicine schedules, identify medicines with low stock, and detect medicines that are nearing their expiry dates. The system also maintains a history of important user activities, making medicine management more organised and reliable.

The project has been developed following the principles of Object-Oriented Programming (OOP) by separating different functionalities into independent packages such as Model, Service, Storage, User Interface, and Data Structure. This modular architecture makes the application easier to understand, maintain, and extend with additional features in the future. The application stores all medicine records locally using text files instead of a database, which keeps the system lightweight while demonstrating the practical implementation of Java File Handling concepts.

1.1 Problem Statement

In the absence of a dedicated medicine management tool, individuals frequently miss medicine doses, are unaware of depleting stock, and unknowingly consume medicines past their expiry date. Manually tracking multiple medicines with different dosage timings, quantities, and expiry dates is error-prone and mentally taxing, particularly for patients managing long-term

or multiple prescriptions. There is a clear need for a simple, offline, desktop-based application that can automatically track medicine inventory, alert users about low stock and upcoming expiries, and remind them of the next medicine to be taken, without requiring an internet connection or a complex database setup.

1.2 Objectives

- To design and develop a desktop application for efficient management of medicine records using Java Swing.
- To implement core data structures (ArrayList, HashMap, Stack, Priority Queue) to optimise storage, searching, undo operations, and reminder scheduling.
- To provide functionality for adding, searching, deleting, and restoring (undo delete) medicine entries.
- To generate automatic alerts for low stock quantities and medicines nearing expiry.
- To implement a Next Medicine Reminder feature based on scheduled dosage timings using a priority queue.
- To maintain a persistent, file-based storage system (medicines.txt and history.txt) that keeps records safe between sessions.
- To maintain a complete activity history of additions, deletions, and restorations for transparency and traceability.
- To design an intuitive, visually improved graphical user interface for ease of use by nontechnical users.

1.3 Project Modules

The Smart Medication System is divided into the following functional modules, each contributing to a specific aspect of medicine management:

- **Dashboard Module** — central navigation hub linking all features of the application.
- **Add Medicine Module** — form-based interface to input and validate new medicine records.
- **Inventory Module** — tabular (JTable) display of all stored medicines with search and delete options.
- **Medicine Service Module** — core business logic for add, delete, search, and restore operations.
- **Reminder & Alerts Module** — Next Medicine Reminder, Low Stock Alert, and Expiry Alert features.
- **File Handling Module** — reading from and writing to medicines.txt and history.txt for persistent storage.
- **History Module** — logs and displays all add, delete, and restore activities performed by the user.
- **Testing & Documentation** — functional/integration testing, Git version control, and report preparation.

1.4 Technologies Used

Category	Technology
Programming Language	Java (JDK 8+)
GUI Framework	Java Swing
Data Structures	ArrayList, HashMap, Stack, Priority Queue
Storage	Text File Handling (medicines.txt, history.txt)
Version Control	Git & GitHub
IDE	IntelliJ IDEA / Eclipse
Design Approach	Object-Oriented Programming (OOP), Modular Package Structure

2. Module Description

This section describes, in detail, the modules implemented by each team member and how they contribute to the overall functioning of the Smart Medication System.

2.1 User Interface Development

Designing and implementing the complete graphical user interface of the application using Java Swing.

- **Dashboard Design:** The Dashboard serves as the central interface of the Smart Medication System. It provides users with easy navigation to all major functionalities of the application, including Add Medicine, Inventory Management, History Tracking, Low Stock Alerts, Expiry Alerts, and Medicine Reminders, through a single unified window.
- **Add Medicine Interface:** A dedicated form was designed to capture medicine name, quantity, dosage time, and expiry date. Input validation was added to prevent empty or incorrectly formatted entries before the record is passed to the Medicine Service layer.
- **Inventory Interface:** A JTable-based view displays all stored medicines in a structured, sortable format, with integrated Search and Delete controls for quick record management.
- **UI Enhancement:** The interface was iteratively refined — including a custom logo placement, styled buttons with hover effects, corrected component alignment, and an accent colour scheme (e.g. exit button in colour #D11A2A) — to improve usability and visual consistency.

2.2 Core Logic and Data Structures

Implementing the application's business logic and the core data structures that give the system its efficiency.

- **Medicine Service:** Implemented Add, Delete, Search, and Restore operations, and synchronised in-memory data with the underlying file storage.
- **Search Optimisation:** A HashMap was used to map medicine names to their records, allowing near-constant time lookups instead of a linear scan through the entire list.

- **Undo Delete:** A Stack was used to record deleted medicines in Last-In-First-Out order, enabling a reliable Undo Delete operation that restores the most recently removed record first.
- **Medicine Reminder:** A Priority Queue, ordered by scheduled dosage time, was used to always retrieve the next medicine due, allowing the Next Medicine Reminder feature to function efficiently as the list of medicines grows.
- **Alerts:** Implemented the logic for Low Stock Alerts (triggered when quantity falls below a defined threshold) and Expiry Alerts (triggered when a medicine is due to expire within 30 days), by comparing stored values against the current system date.

2.3 Storage, History and Software Engineering

For persistent storage, activity tracking, testing, and project documentation.

- **File Handling:** Implemented reading and writing of medicine records to medicines.txt, including logic to rewrite the file cleanly after a deletion and to prevent duplicate entries.
- **History Module:** Implemented activity logging to history.txt, recording every Added, Deleted, and Restored action along with a timestamp, and displaying the complete history within the application.
- **Exception Handling:** Added validation and error handling around file I/O and user input to prevent the application from crashing on malformed data.
- **Testing:** Conducted functional and integration testing across all modules, identifying and resolving bugs such as unresponsive buttons and formatting issues in exported text data.
- **Version Control:** Managed the Git/GitHub repository, including commit history, branching, and merging, and prepared the README documentation, screenshots, and this project report.

3. Implementation

This section explains how the project was actually built — first the data structures that were chosen and why, and then the key pieces of source code, explained step by step so the reasoning behind each line is clear and easy to present during the viva.

3.1 Data Structures / Algorithms Used

Four data structures form the backbone of the Smart Medication System. Each one was chosen to solve one specific problem efficiently, rather than using a single generic structure for everything. The table and explanations below summarise the choice, the reasoning, and the resulting performance.

Data Structure	Used For	Time Complexity
ArrayList	Master storage of all medicine records	$O(1)$ access, $O(n)$ search
HashMap	Fast lookup of a medicine by name	$O(1)$ average lookup
Stack	Undo Delete (restore last deleted medicine)	$O(1)$ push / pop
Priority Queue	Next Medicine Reminder (earliest dose first)	$O(\log n)$ insert / remove

1. ArrayList

The ArrayList (medicineList) is the single master list holding every medicine record currently in the system. It was chosen over a plain array because it resizes automatically as medicines are added or removed, which is essential since the number of medicines is never fixed. Every other data structure in the project works alongside this list — the HashMap indexes it, the Stack temporarily holds items removed from it, and the Priority Queue reorders a copy of it for reminders.

2. HashMap

The HashMap (medicineMap) stores each medicine's name as the key and the medicine object as the value. Without it, finding a medicine by name would require scanning the entire ArrayList one item at a time — an $O(n)$ operation. With the HashMap, the same lookup happens in $O(1)$ time on average, which keeps the Search and Delete features fast even as the inventory grows to dozens or hundreds of entries.

3. Stack

The Stack (deletedStack) exists purely to support the Undo Delete feature. A stack follows Last-In-First-Out (LIFO) ordering — the last medicine pushed onto it is the first one popped off. This matches exactly how 'undo' is expected to behave: if a user deletes Medicine A and then Medicine B, pressing Undo should bring back Medicine B first, not Medicine A. A Stack was

chosen specifically because no other structure enforces this reverse-chronological order as naturally or efficiently.

4. Priority Queue

The Priority Queue (reminderQueue) automatically keeps medicines sorted by their scheduled dosage time, without the program needing to manually re-sort a list every time a medicine is added. A custom Comparator tells the queue how to compare two medicines (by dosage time), so calling peek() always returns whichever medicine is due soonest. This is far more efficient than repeatedly sorting the entire ArrayList ($O(n \log n)$ every time) — insertion and retrieval here cost only $O(\log n)$.

3.2 Important Source Code with Explanation

The four snippets below are taken from the Medicine Service layer, which contains the core business logic of the application. Each snippet is followed by a plain-language, step-by-step explanation of what the code does and why it was written that way.

(a) Adding a Medicine and Updating the HashMap Index

```
public void addMedicine(Medicine medicine) {
    medicineList.add(medicine);
    medicineMap.put(medicine.getName(), medicine);
    fileHandler.saveMedicines(medicineList);
    historyLogger.log("Added", medicine.getName());
}
```

Fig. 3.1 — addMedicine() method

Explanation — what each line does:

Step 1: medicineList.add(medicine) — the new medicine object is appended to the master ArrayList, so it becomes part of the in-memory inventory.

Step 2: medicineMap.put(...) — the same medicine is also indexed in the HashMap using its name as the key, so it can be found instantly later without searching the whole list.

Step 3: fileHandler.saveMedicines(...) — the updated list is written to medicines.txt immediately, so the record is not lost if the application closes.

Step 4: historyLogger.log("Added", ...) — an entry is recorded in the History module noting that this medicine was added, along with a timestamp.

In short, one method call keeps four things in sync at once: the list, the search index, the saved file, and the activity history — which is why every Add action stays consistent across the whole application.

(b) Undo Delete using a Stack

```
public void deleteMedicine(String name) {
    Medicine removed = medicineMap.remove(name);
    medicineList.remove(removed);
    deletedStack.push(removed); // LIFO storage
    fileHandler.saveMedicines(medicineList);
}
```



```

public void undoDelete() {    if (!deletedStack.isEmpty()) {
    Medicine restored = deletedStack.pop();
    addMedicine(restored);
    }
}

```

Fig. 3.2 — *deleteMedicine()* and *undoDelete()* methods

Explanation — what each line does:

Step 1: `medicineMap.remove(name)` — the medicine is located instantly via the `HashMap` and removed from the index; its object reference is kept in the variable `removed`.

Step 2: `medicineList.remove(removed)` — the same object is removed from the master `ArrayList` so it no longer appears in the inventory.

Step 3: `deletedStack.push(removed)` — instead of discarding the deleted medicine, it is pushed onto the `Stack`, preserving it in case the user wants it back.

Step 4: In `undoDelete()`, `deletedStack.pop()` removes and returns the most recently deleted medicine (LIFO order), which is then passed back into `addMedicine()` to restore it fully — back in the list, the index, the file, and the history.

This two-method pair is a clean example of how a `Stack` solves the 'reverse the last action' problem far more simply than manually tracking deletion order in a list.

(c) Next Medicine Reminder using a Priority Queue

```

PriorityQueue<Medicine> reminderQueue = new
PriorityQueue<>(Comparator.comparing(Medicine::getDosageTime));

public Medicine getNextMedicine() {    return
reminderQueue.peek();    // earliest scheduled dosage
}

```

Fig. 3.3 — *Priority Queue reminder logic*

Explanation — what each line does:

Step 1: `new PriorityQueue<>(Comparator.comparing(Medicine::getDosageTime))` — creates a queue that automatically keeps medicines ordered by their dosage time, from earliest to latest, every time one is added.

Step 2: `getNextMedicine()` calls `reminderQueue.peek()` — this looks at (without removing) the medicine at the front of the queue, which is always the one due soonest.

Because the queue maintains this order internally, the application never has to loop through every medicine to find the next one — it simply asks the queue, which is both faster and simpler to code correctly.

(d) Low Stock and Expiry Alert Logic

```

public boolean isLowStock(Medicine m) {
    return m.getQuantity() < 5;
}

public boolean isNearExpiry(Medicine m) {
    long daysLeft = ChronoUnit.DAYS.between(LocalDate.now(),
m.getExpiryDate());    return daysLeft <= 30;
}

```

Fig. 3.4 — *Alert-checking methods*

Explanation — what each line does:

Step 1: `isLowStock(m)` simply compares the medicine's current quantity to a fixed threshold (5). If the quantity has dropped below that, the method returns true and the UI flags it as low stock.

Step 2: `ChronoUnit.DAYS.between(...)` calculates the number of days between today's date and the medicine's expiry date.

Step 3: `isNearExpiry(m)` returns true if that gap is 30 days or less, meaning the medicine is close enough to expiry to warn the user.

Both methods are called for every medicine each time the Dashboard or Alerts screen is opened, so the alerts always reflect the current date and current stock — not a stale, previously calculated value.

4. Screenshots of the Project

This section presents the major interfaces of the Smart Medication System along with a brief description of each.

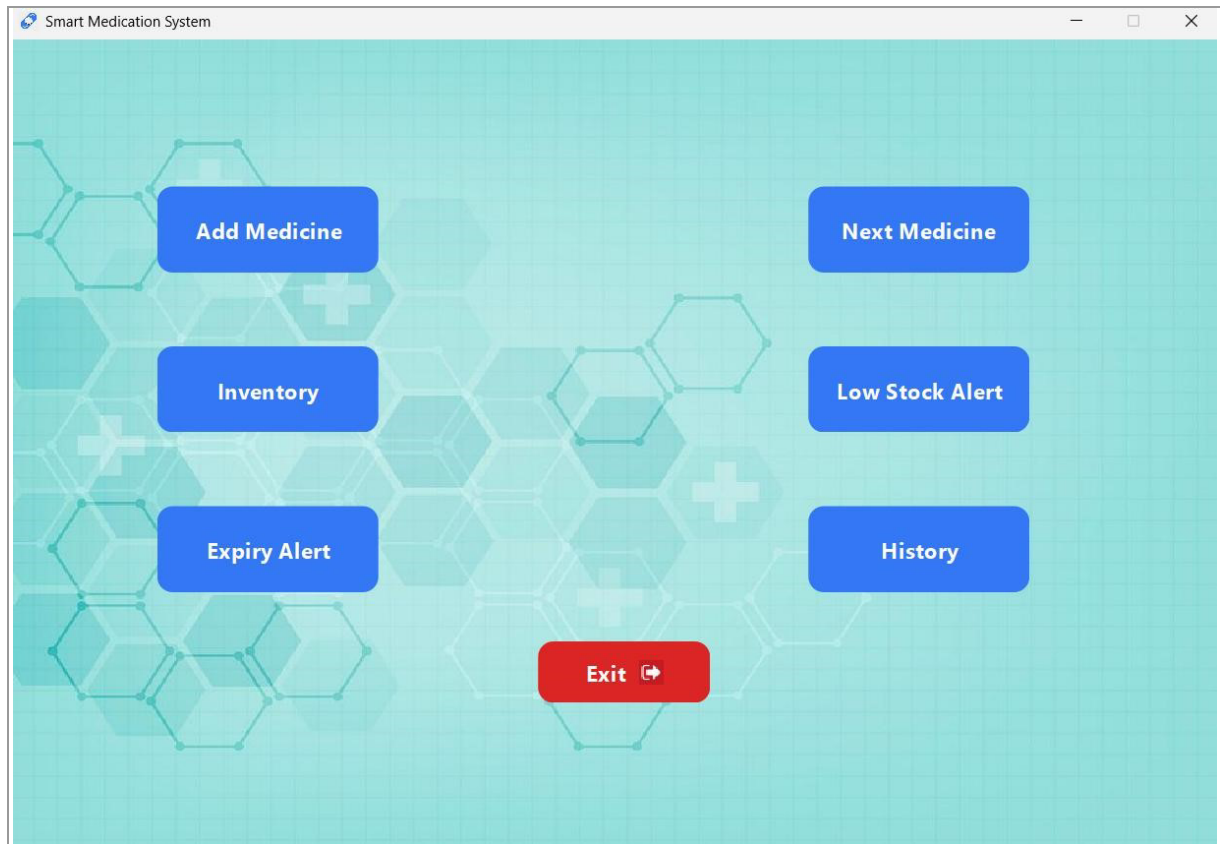


Fig. 4.1 — Dashboard: central navigation hub providing access to Add Medicine, Inventory, History, Reminders, and Alerts.

Smart Medication System — Project Report

Add Medicine

Medicine Name

Carbamazepine 200mg

Dosage

1 Tablet

Time

08:00 AM

Quantity

60

Expiry Date

01/11/2028

Save Medicine

Fig. 4.2 — Add Medicine: form used to enter a new medicine's name, quantity, dosage time, and expiry date, with input validation.

Medicine Inventory

Medicine Name

Search

Delete

Undo Delete

Refresh

Name	Dosage	Time	Quantity	Expiry Date
Dolo	300mg	09:00 PM	2	02-07-2027
Ibuprofen	400mg	02:00 PM	20	20-12-2027
Crocin	500mg	08:00 PM	10	01-01-2028
Paracetamol	500mg	10:00 PM	4	15-07-2026
Gabapentin 300mg	1 capsule	08:00 PM	8	08/12/2027
Carbamazepine 200mg	1 Tablet	08:00 AM	60	01/11/2028

Fig. 4.3 — Inventory: JTable view of all stored medicines with integrated Search and Delete controls.

Message

Next Medicine Due
Ibuprofen
02:00 PM

OK

Fig. 4.4 — Reminder view showing the next scheduled medicine, generated using the Priority Queue.

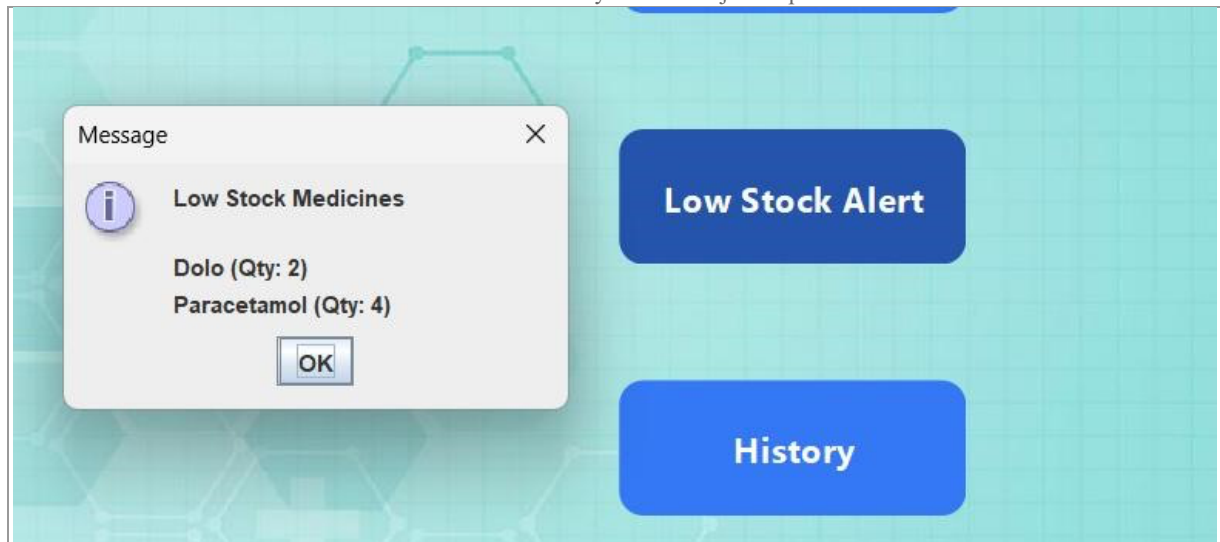


Fig. 4.5 — Alerts panel highlighting medicines that are low in stock or nearing expiry.

Activity History		
Action	Medicine	Date Time
Added	Carbamazepine 200mg	10-07-2026 10:55:38
Deleted	Gabapentin 300mg	10-07-2026 10:54:08

Fig. 4.6 — History: chronological log of Added, Deleted, and Restored medicine actions.

5. Future Scope of the Project

While the current version of the Smart Medication System fulfils its core objectives as a standalone desktop application, several enhancements can be introduced in the future to increase its scope and usability:

- **Database Integration:** Replacing text-file storage with a relational database (e.g. MySQL or PostgreSQL) to support larger datasets and multi-user access.
- **Mobile Application:** Extending the system to Android/iOS so users can manage medicines and receive reminders on the go.

- **Push/SMS Notifications:** Integrating real-time notification services so reminders reach the user even when the application is closed.
- **Cloud Synchronisation:** Storing data in the cloud to allow access across multiple devices and to prevent data loss.
- **Doctor/Caretaker Dashboard:** Adding a secondary login for caretakers or doctors to remotely monitor a patient's medication adherence.
- **Barcode/OCR Scanning:** Allowing medicines to be added automatically by scanning the packaging instead of manual entry.
- **Analytics:** Introducing usage analytics and adherence reports to help users track their medication habits over time.

Conclusion

The Smart Medication System successfully demonstrates the practical application of core computer science concepts — including Object-Oriented Programming, Java Swing-based GUI design, file handling, and fundamental data structures — in solving a real-world healthcare problem. By combining an ArrayList for dynamic storage, a HashMap for fast searching, a Stack for reliable undo operations, and a Priority Queue for efficient reminder scheduling, the project achieves a balance between simplicity and technical depth.

Through this project, the team gained hands-on experience in modular software design, collaborative development using Git and GitHub, and the practical trade-offs involved in choosing appropriate data structures for a given problem. Beyond fulfilling the academic requirements of the course, the Smart Medication System provides a genuinely usable tool for everyday medicine management and lays a solid foundation for future enhancement into a fullfledged, database-backed, cross-platform healthcare assistant.

References

- ❑ Oracle Java Documentation — <https://docs.oracle.com/javase/8/docs/api/>
- ❑ Java Swing Tutorial — Oracle: The Java Tutorials, "Creating a GUI with JFC/Swing"
- ❑ GeeksforGeeks — Java Collections Framework (ArrayList, HashMap, Stack, PriorityQueue)
- ❑ GitHub Documentation — <https://docs.github.com/>
- ❑ Course lecture notes and lab material, Department of Computer Science and Engineering, Lovely Professional University
- ❑ Stack Overflow — Community discussions on Java Swing, Collections, and File I/O (<https://stackoverflow.com/>)